

From small models to frontier models, and prompting as engineering

Friday's model and the frontier assistant in your browser share an architecture. What separates them is scale — of parameters, data and compute — and two stages of training after the first. Understanding those differences is what turns prompting from superstition into engineering: anatomy, context, iteration, and verification as a habit.

Unit 6 · Frontier

80 minutes

Lab 18 · five prompt revisions

Your AI Tools lecture, part 1

The leap

Prompt anatomy

Iteration

Lab 18

Before Friday — the last teaching session

PART 1 · 25 MINUTES

The leap: same architecture, a different world

1

A frontier model is still a stack of transformer blocks predicting the next token. Three differences, each explaining something specific that Friday's model could not do:

Stage	What happens	Why Friday's model lacks it
Pretraining at scale	next-token prediction on a vast, filtered corpus, with far more parameters	distilgpt2 is small and saw much less text, so its knowledge and coherence are thin
Instruction tuning	further training on examples of instructions paired with good responses	it continues text; it was never taught that a question wants an answer
Preference training (RLHF and relatives)	people — and increasingly models — compare responses; the model is trained towards the preferred ones	nothing pushed it towards helpful, honest or safe responses

SCALING LAWS

Across many experiments since about 2020, a language model's pretraining loss has fallen smoothly and predictably as parameters, data and compute grow together — a power law, not a sudden jump. Later work showed that data must grow with model size: a large model trained on too little text is wasted compute. **Predictability is what justified spending so much on each new generation.**

How far apart are the two ends? Frontier developers rarely publish exact figures, but training compute differs from a laptop-scale model by roughly six orders of magnitude or more. For scale, European AI regulation presumes a general-purpose model trained with more than 10^{25} floating-point operations carries systemic risk.



Scale does not remove the mechanism

A frontier model still generates the likely next token. It can produce a fluent, confident, false citation for exactly the reason Friday's model produced fluent nonsense — only far more rarely and far more convincingly. That single fact is why verification is part of the method, not an afterthought.

PART 2 · 20 MINUTES

Prompt anatomy

Six ingredients, each doing one job.

2



Task and goal

What to produce and what it is for. *"So they can explain a forward pass to a classmate."*



Audience

Who reads it, and what they already know.



Context



Constraints

The material the answer must use — notes, data, code, the course map.

Scope, length, what to leave out, what must not be invented.



Output format

Sections, a table, code with tests, a JSON schema.



Verification

What to flag as uncertain; what the reader should check, and how.

CONTEXT WINDOWS

Everything the model can use — instructions, pasted documents, the conversation so far — must fit in its context window, measured in tokens. Windows are now large, but not free: long, unfocused context can bury the one instruction that matters. **Give the model what the task needs, not everything you have.**

PART 3 · 15 MINUTES

Iteration: improvement measured, not felt

3

Treat a prompt like code. Change **one** ingredient at a time, run it, score the output against a rubric decided in advance, and keep the change only if the score rises. Then remove what does not earn its place: a better prompt is not a longer prompt.

Revision	Adds	Typical effect
0	"Explain neural networks."	generic, pitched at nobody
1	audience and goal	right level, right purpose
2	context: what the reader already knows	stops re-explaining the basics
3	constraints and scope	shorter, on topic
4	structure and examples	usable, checkable
5	verification	says what to check and where it is unsure

VERIFICATION AS A HABIT, NOT A WARNING

For any output you will rely on: **run the code**, with a test that could fail; **check one fact** against a source you trust; **ask for the working** so errors are visible. The course rule since session 4 — use these tools for lab code if you can explain every line — is this habit, formalised.

LAB 18 · 25 MINUTES IN CLASS

Five prompt revisions

4

Lab 18 — `CSC4315_Lab18_Five_Prompt_Revisions`. One frontier model, one weak prompt, five documented revisions scored on a fixed rubric, and one sentence deleted at the end.

Before Friday — the last teaching session

1. **Finish Lab 18.**
2. **Think about this:** the assistant does not know your course documents. What would it need, beyond a better prompt, to answer questions about them reliably?



Session 28 — Friday 11 December, 11:30

Agents, tools and retrieval — and where this course ends.

CSC 4315 — Artificial Intelligence · Session 27 of 30 · Fall 2026

Prof. Guillaume Sanje Mpacko, Ph.D. · School of Science and Engineering · International University of Grand-Bassam